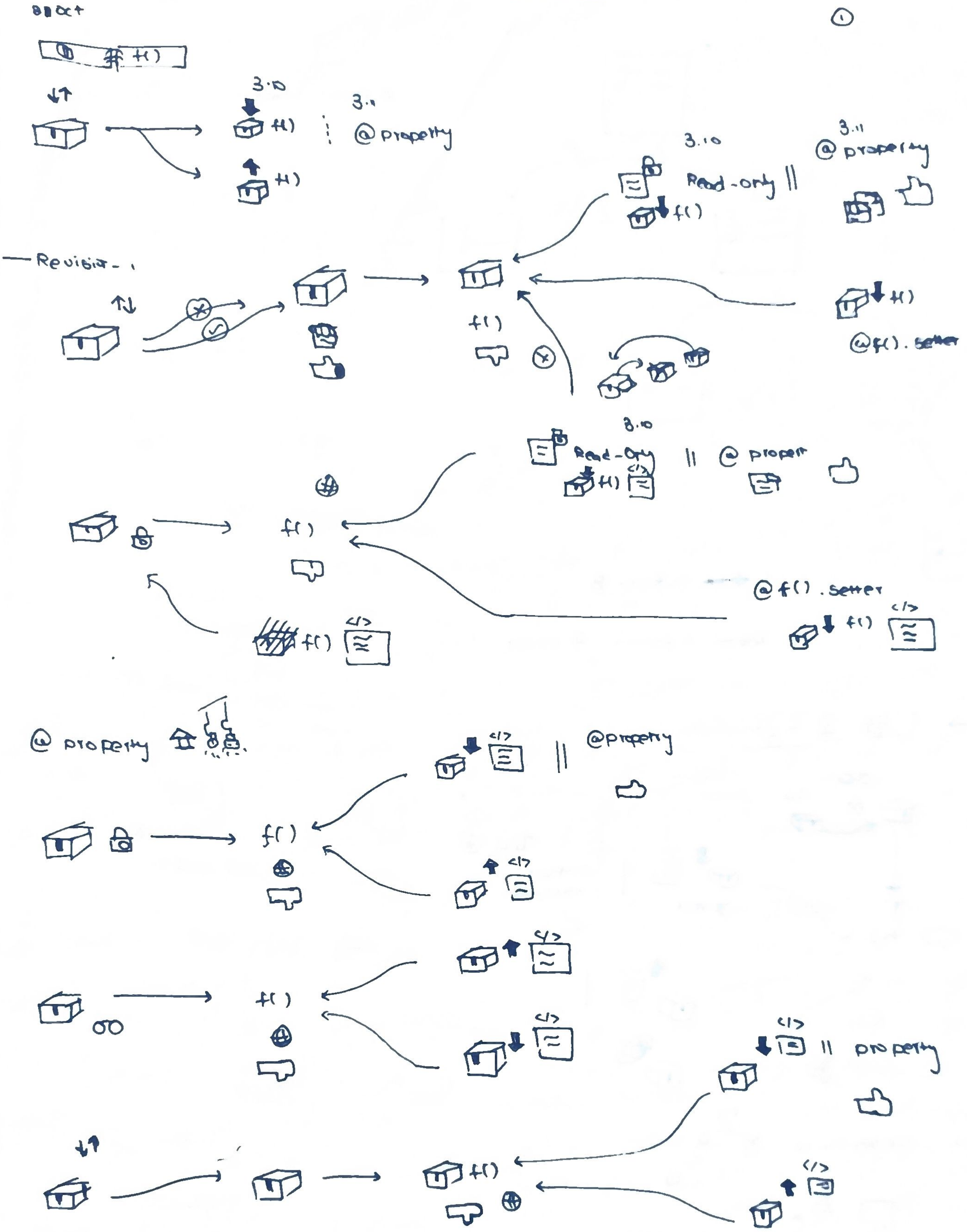
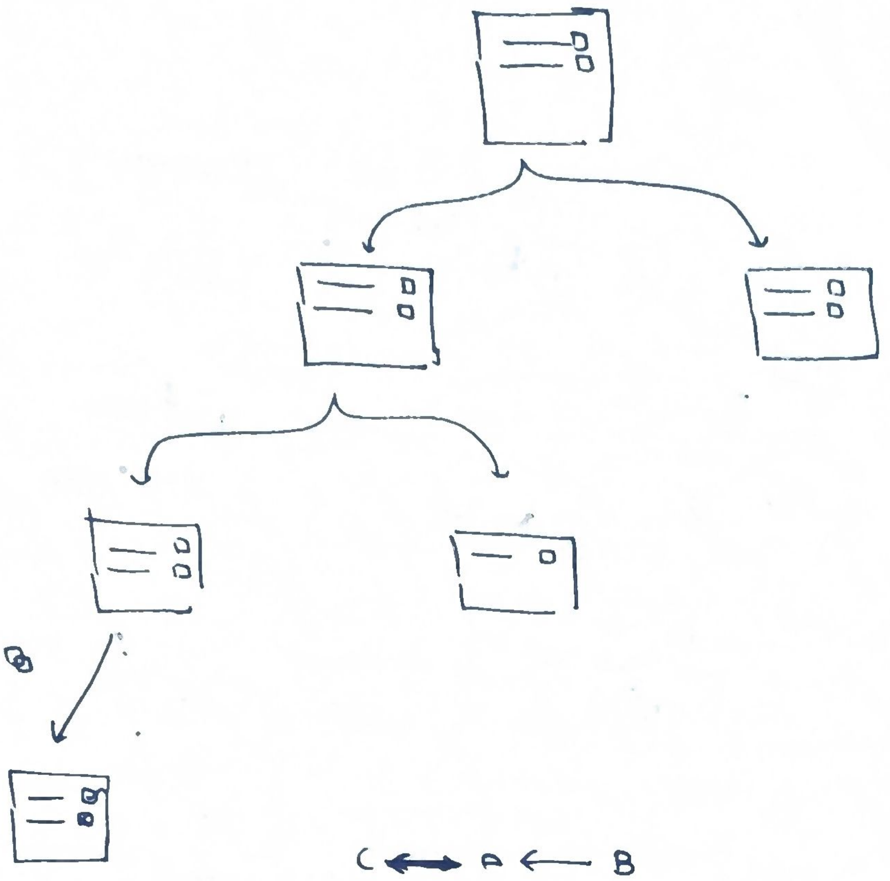
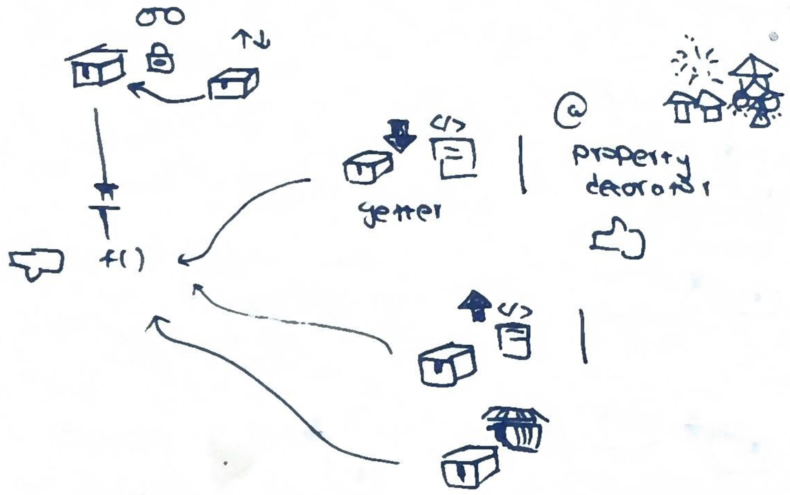

object
f()
3.0
f()
f()
3.1
@property
3.10
Read-only ||
f()
3.11
@property
Revisit - 1
f()
f()
@f().setter
3.10
Read-Only
f()
</>
|| @propert
f()
@f().setter
f()
</>
f()
</>
@property
</>
|| @property
f()
</>
</>
f()
</>
f()
</>
</>
|| property
f()
</>



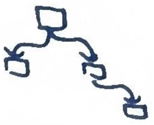
parent class

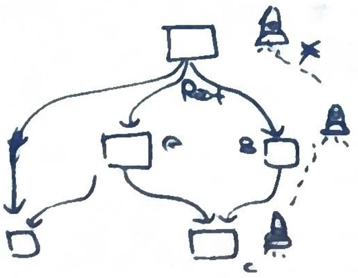
C ⟷ A ← B

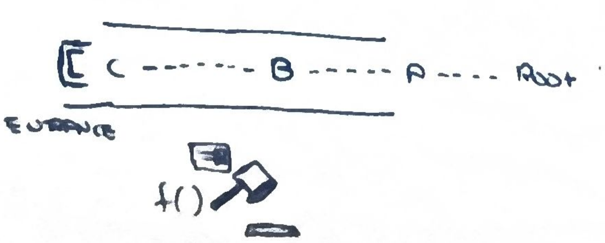
D ← A ← C # error

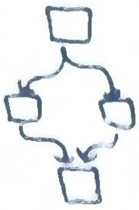

getter
@
property decorator
f()

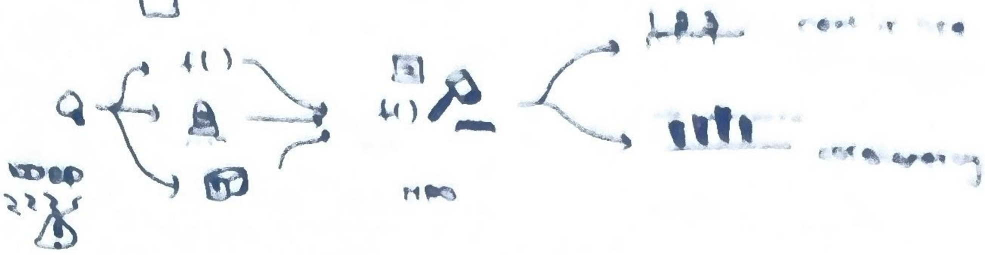
→ Inheritance

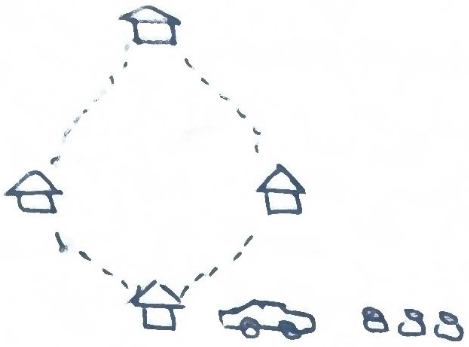




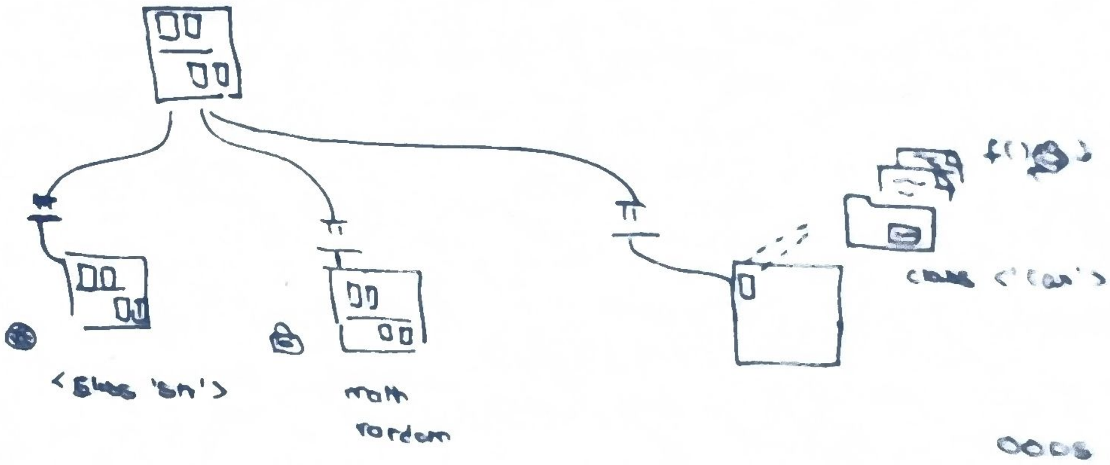

C
B
A
Root
ENTRANCE
f()




f()
f()
MRO
code reusing

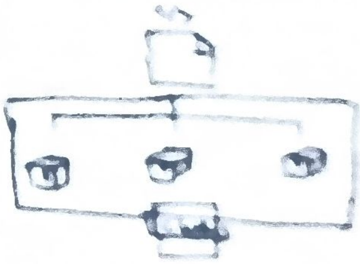


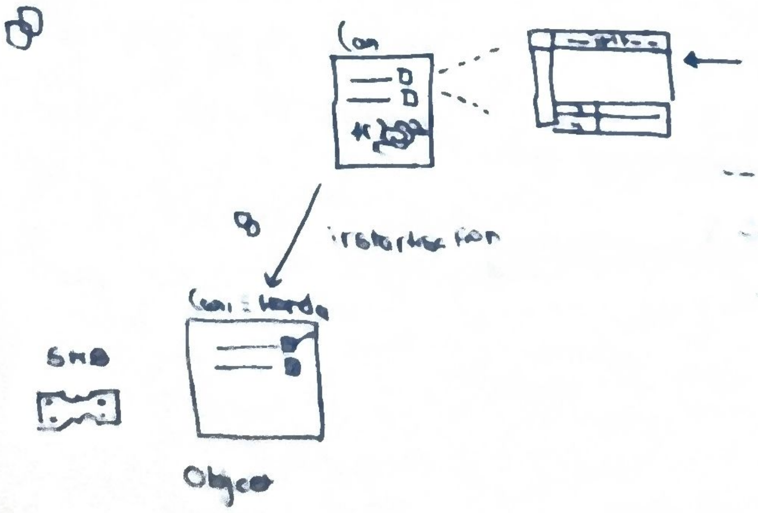
Revision 1


f()
class <'car'>
<class 'str'>
math
random

class == Data type

OOPS




car
instantiation
car = honda
SMB
Object

__init__ constructor

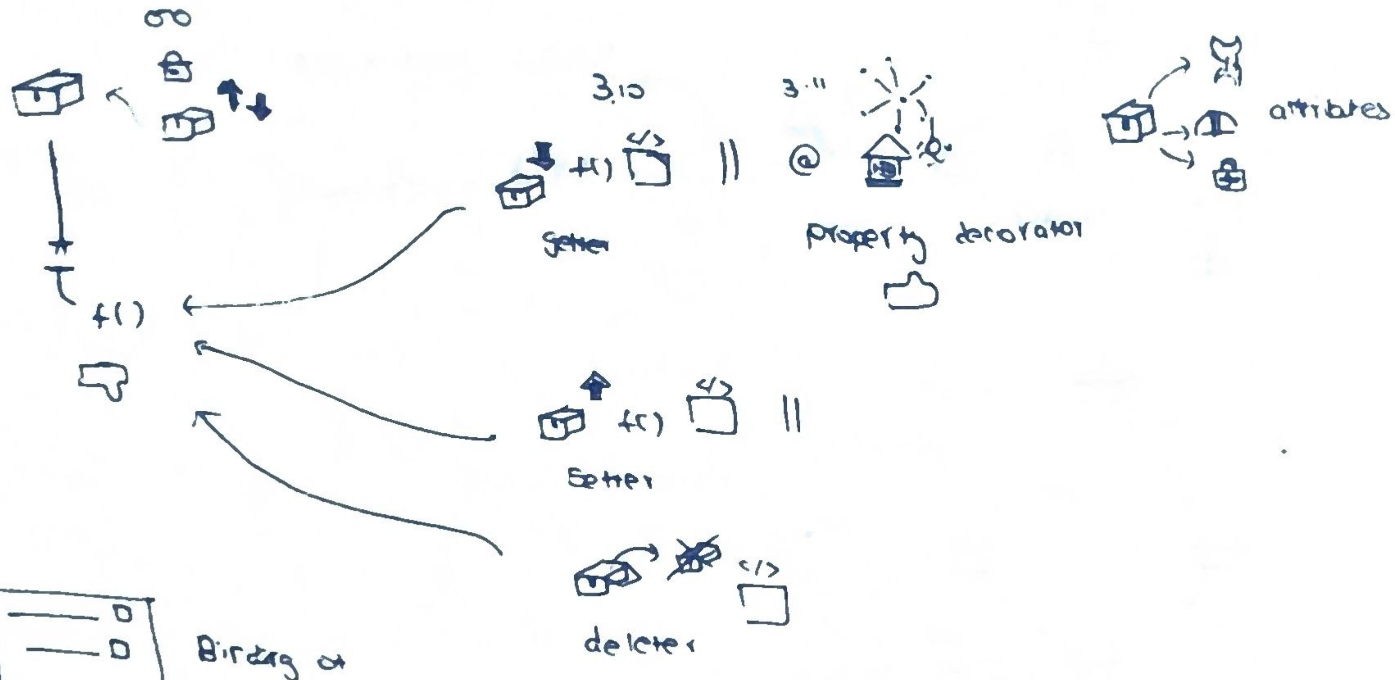
__del__ Destructor

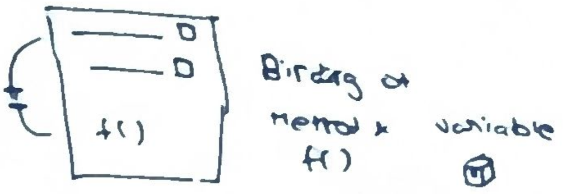
__str__

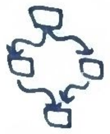
4 pillars of OOPS

f()
abstraction
Encapsulation

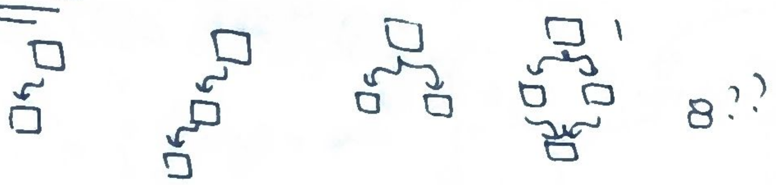
Encapsulation

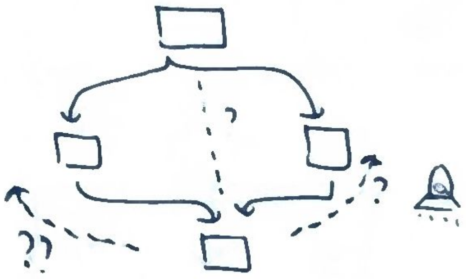

3.10
f()
</>
||
getter
3.11
@
property decorator
attributes
f()
</>
||
setter
</>
deleter

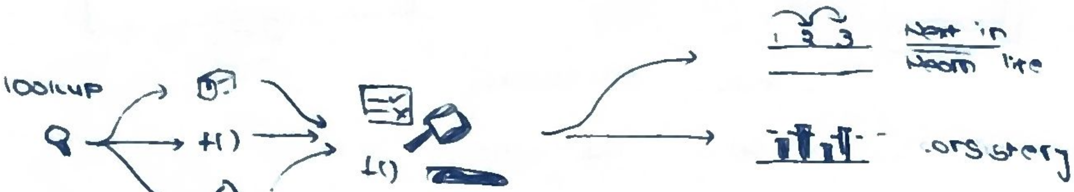

f()
Binding of
method &
f()
variable



Inheritance





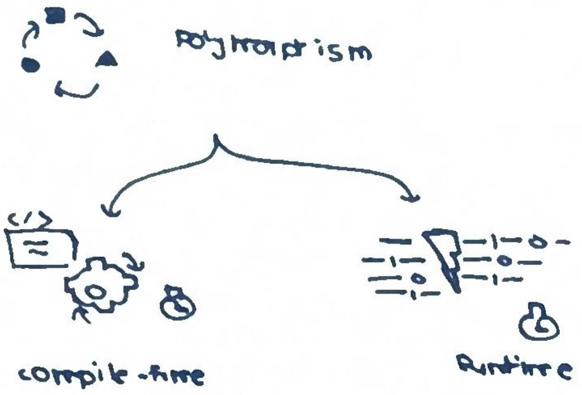

lookup
f()
f()
1 2 3
Next in
Room line
consistency

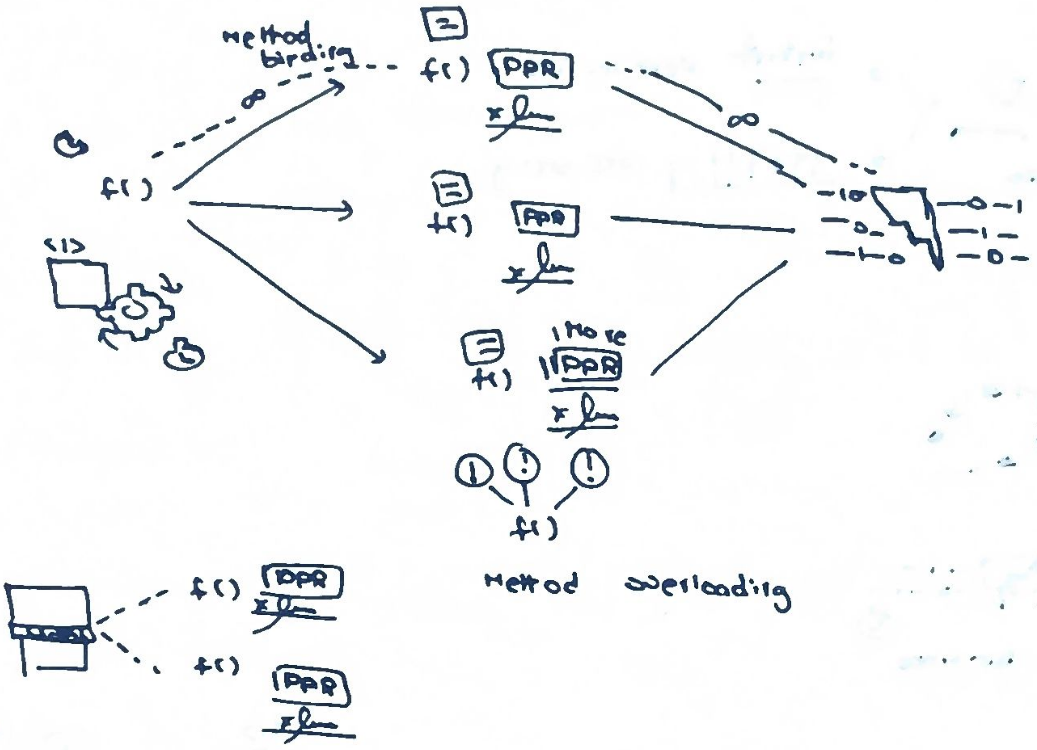
MRO
(method Resolution
order)

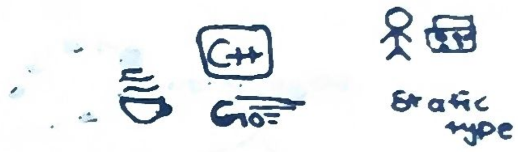
Dunder methods   Class Name __mro__ - -

22 October

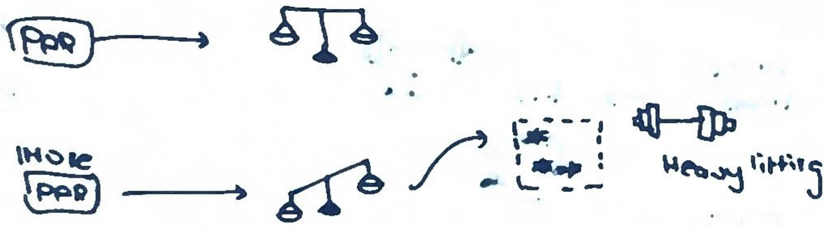
①

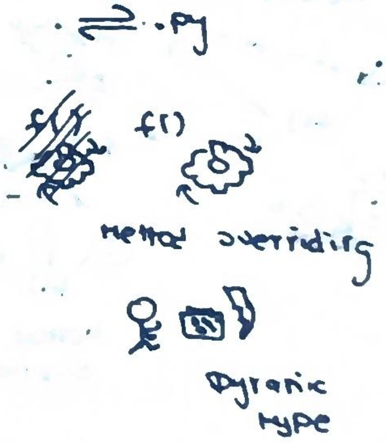

Polymorphism
</>
compile-time
Runtime

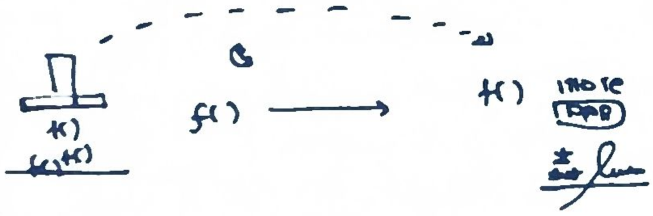

method binding
∞
f()
PPR
∞
</>
f()
PPR
f()
more
PPR
f()
Method overloading
f()
PPR
f()
PPR


C++
Go
Static type

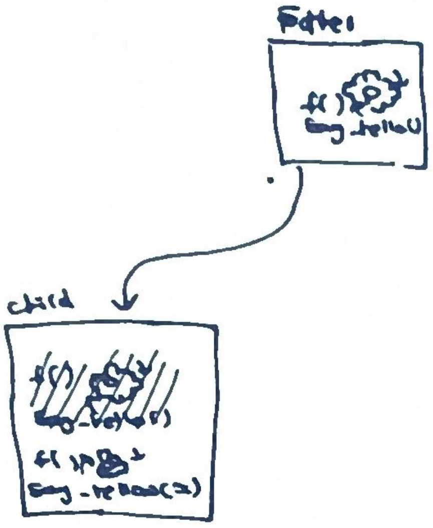
xx

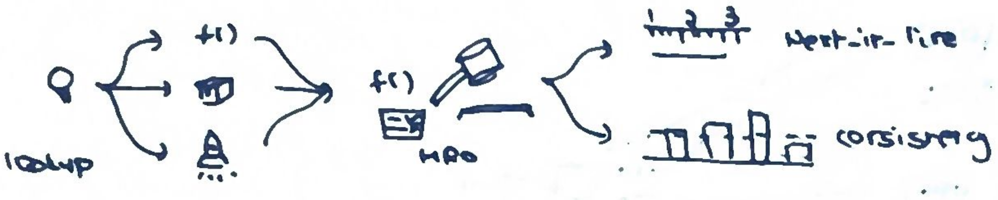

PPR
more
PPR
Heavy lifting


.py
f()
f()
method overriding
Dynamic type

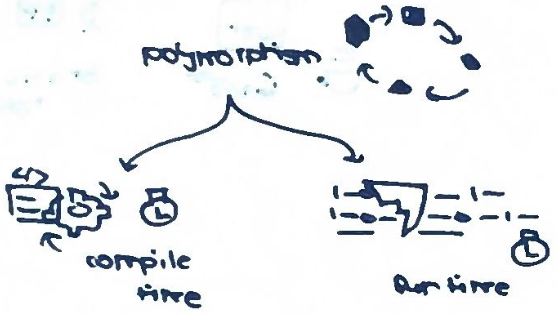

f()
f(x)
f()
f()
more
PPR

one function f() take many forms depends on more PPR

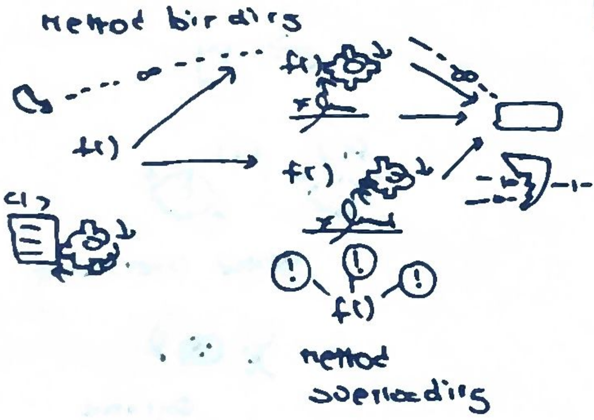

Father
f() say_hello()
child
~~f() say_hello()~~
f() say_hello(x)

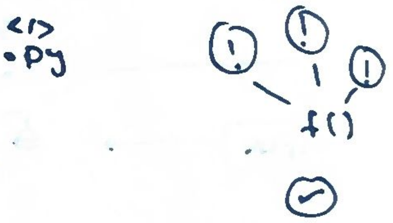

lookup
f()
f()
MRO
1 2 3
next-in-line
consistency

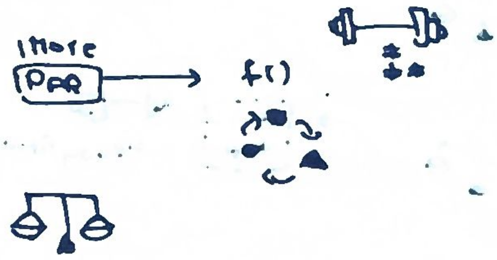
Revision-2

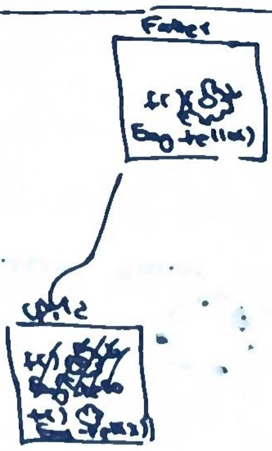

polymorphism
</>
compile time
Run time

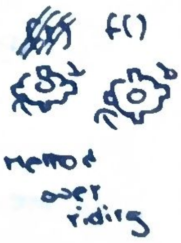
C++
Go

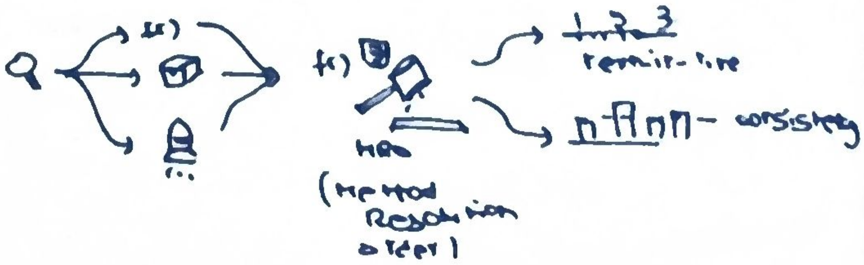

method binding
f()
f()
f()
</>
f()
method overloading


</>
.py
f()

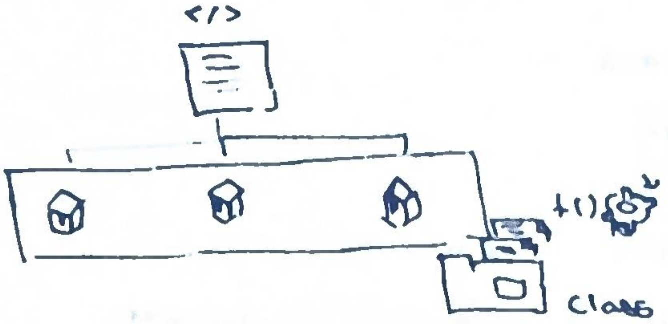

more
PAR
f()


Father
f() say_hello()
child


f()
method overriding

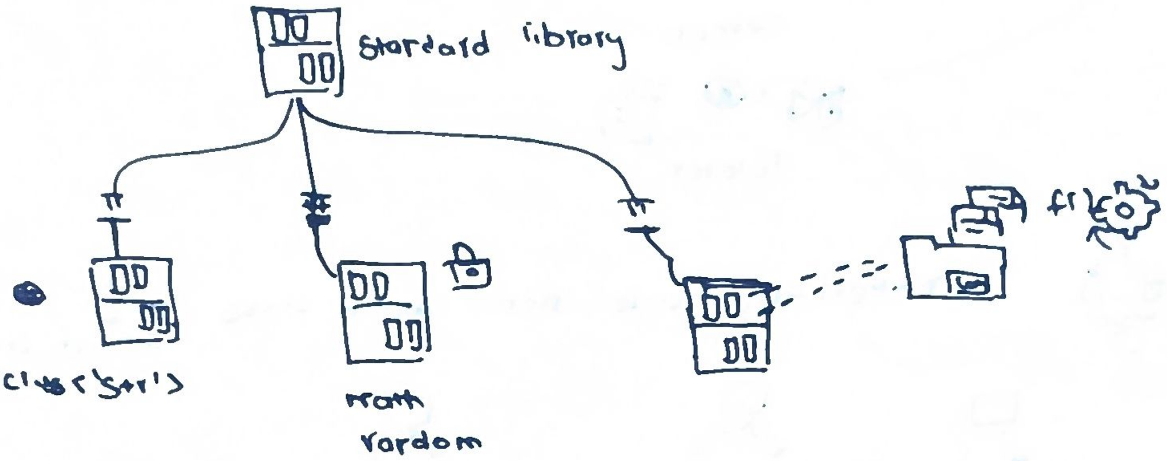

f()
f()
MRO
(Method Resolution Order)
1 2 3
remain-line
consistency

OOPS



program organised with objects

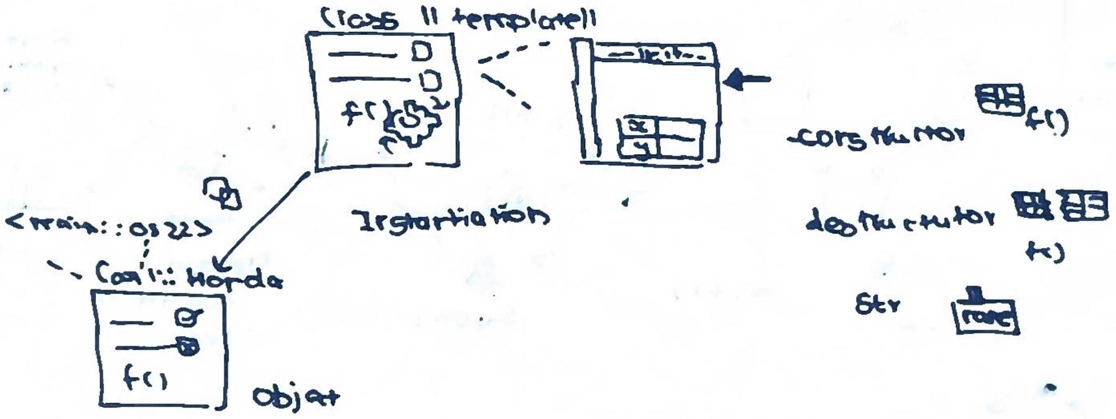
Class class == Data type



Standard library
f()
class('str')
math
random

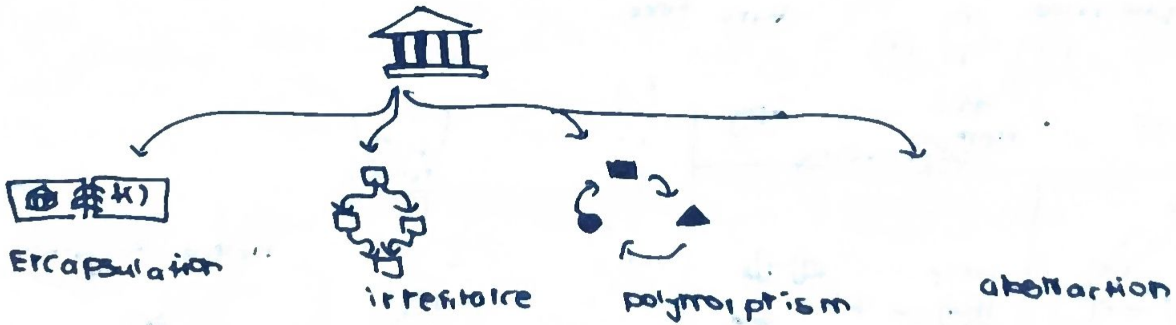
Class is way of creating our own Data type

Syntax ::



class "template"
f()
Instantiation
constructor
f()
destructor
f()
str
<main::0322>
Car:: Honda
f()
object

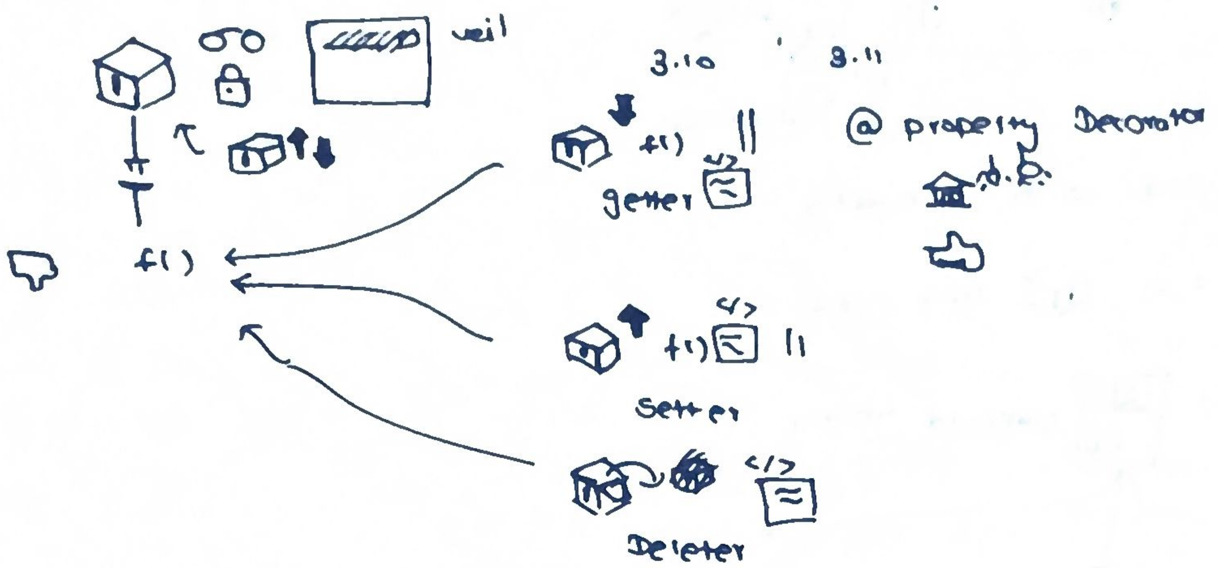
Instance of a class is object
(using the template)



f()
Encapsulation
inheritance
polymorphism
abstraction

Encapsulation

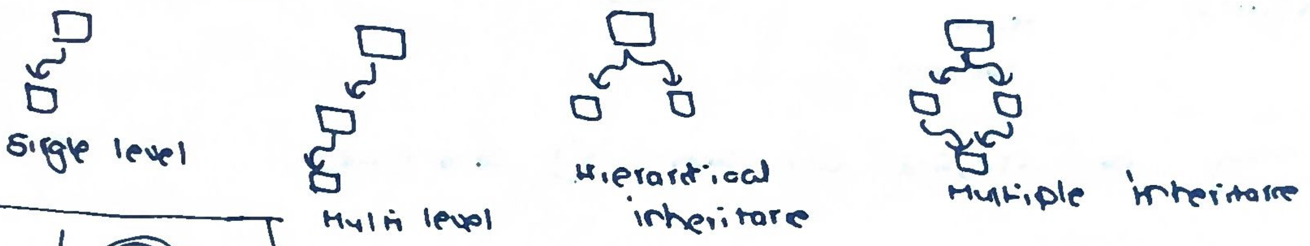
- Binding of variable & method


veil
f()
3.10
3.11
f()
getter
@property Decorator
f()
setter
Deleter

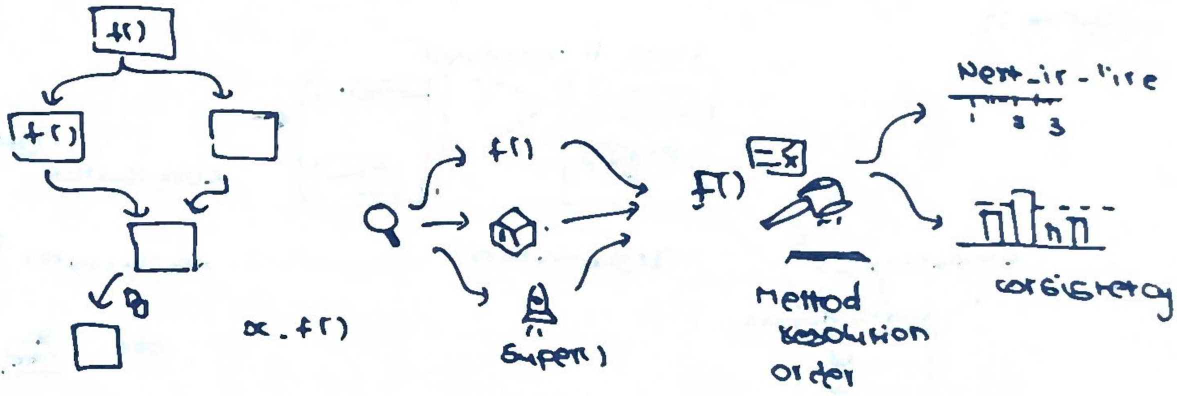
~~Polymorphism~~ Inheritance

Inheriting code from parent class

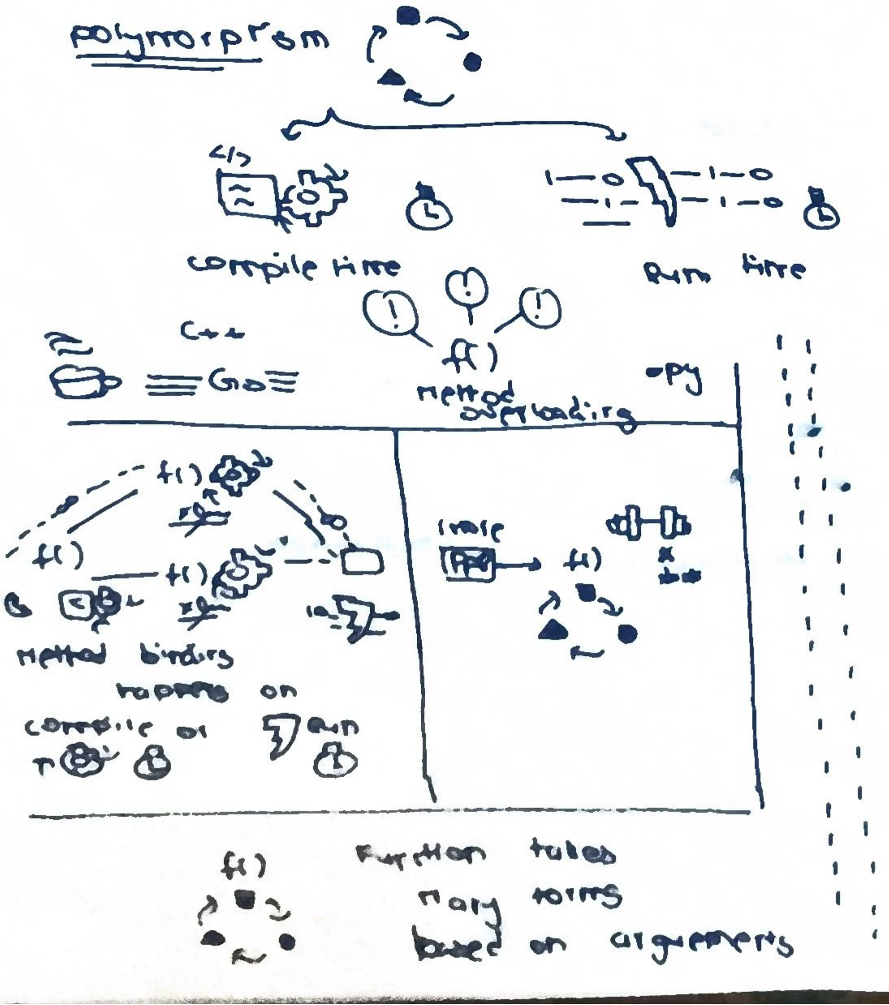
Reusability

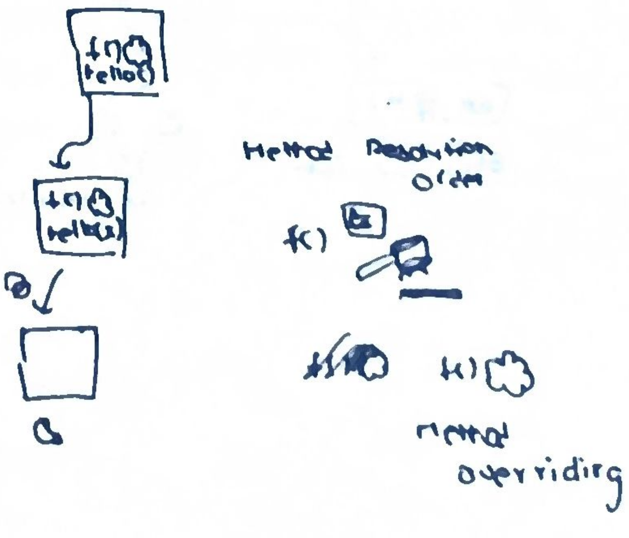

Single level
Multi level
Hierarchical inheritance
Multiple inheritance

Golden Rule
Don't Reinvent the wheel

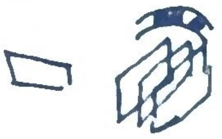

f()
f()
x.f()
f()
super()
f()
Method Resolution order
Next-in-line
1 2 3
consistency

Polymorphism

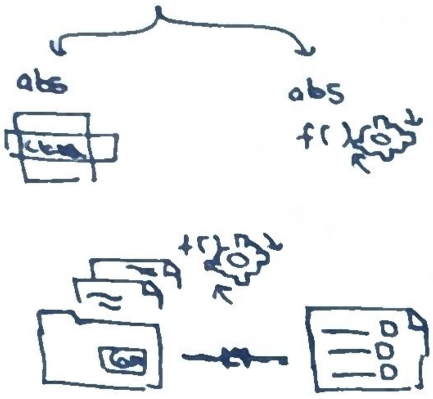

compile time
Run time
C++
Go
f()
method overloading
.py
f()
f()
f()
Method binding happens on compile or Run
Inline
f()
f()
Function takes many forms based on arguments

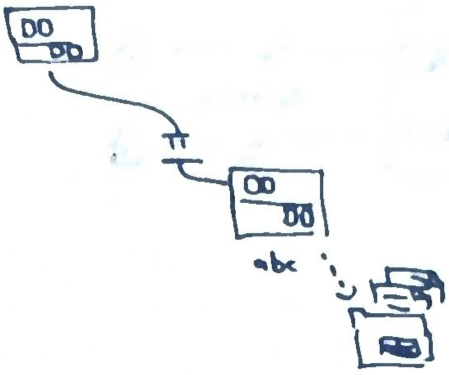

f()
hello()
f()
hello(x)
Method Resolution order
f()
f()
Method overriding

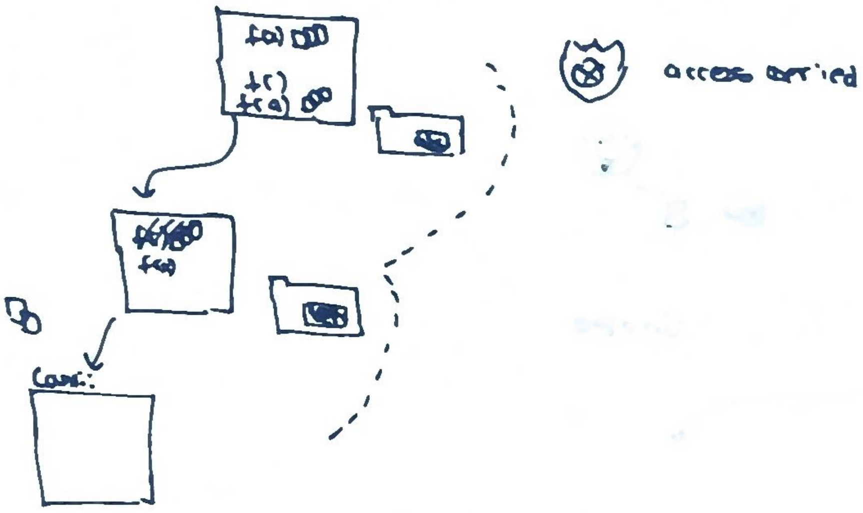
22 Oct



abstraction :: hides the unnecessary Details



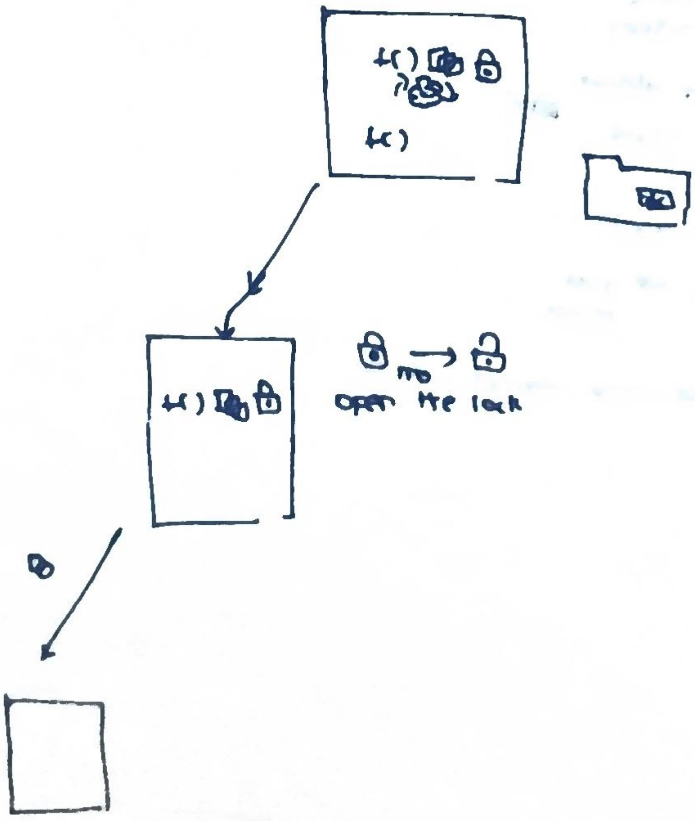



f(a)
f()
f(a)
f(a)
Cust:
access denied

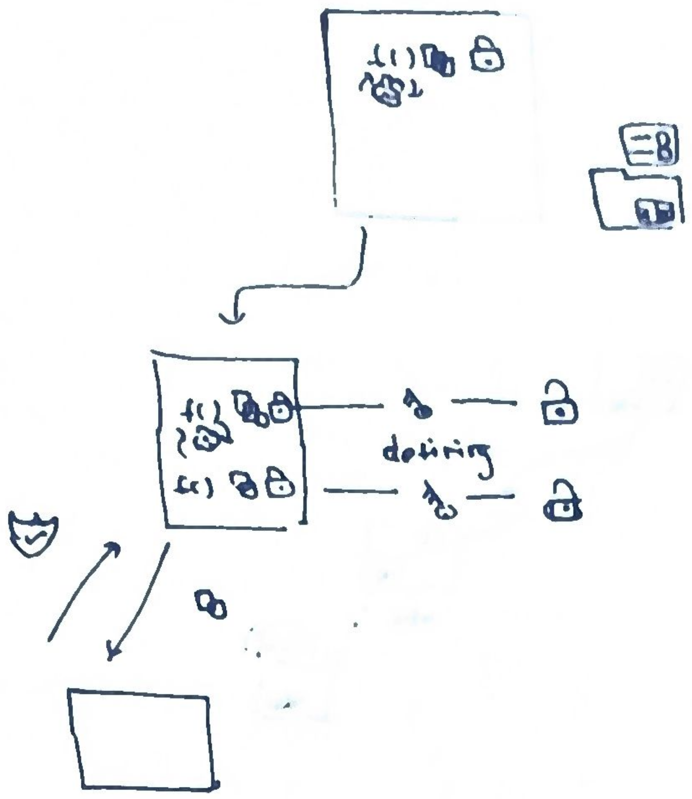
call. f(x) ✓ vehile ~~[illegible]~~abc

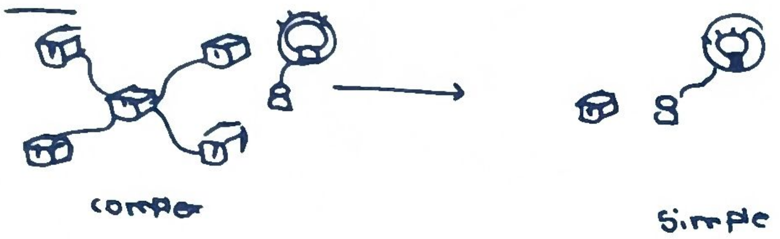
call. f() ✓ vehile ← abc

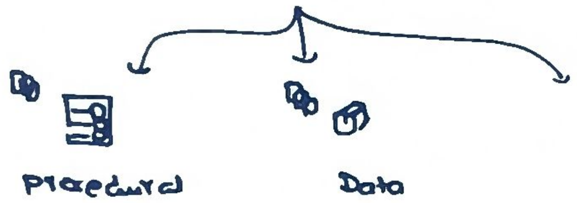
call. f(a) ⊗

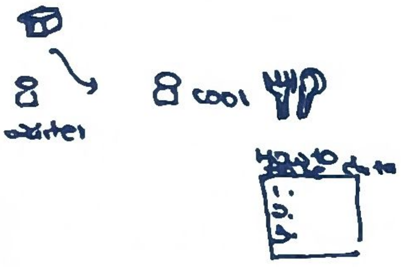

f()
f()
f()
no
open the lock

abstraction

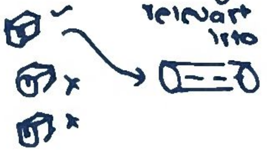



complex
simple


Procedural
Data


waiter
cool
How to [illegible] data
1.
2.
3.

Drawing only relevant info into order pipeline



Making a cake order

- only care about size & type

- who you are?
where are you from

Person → waiter

Describing an event that has specific function / instruction